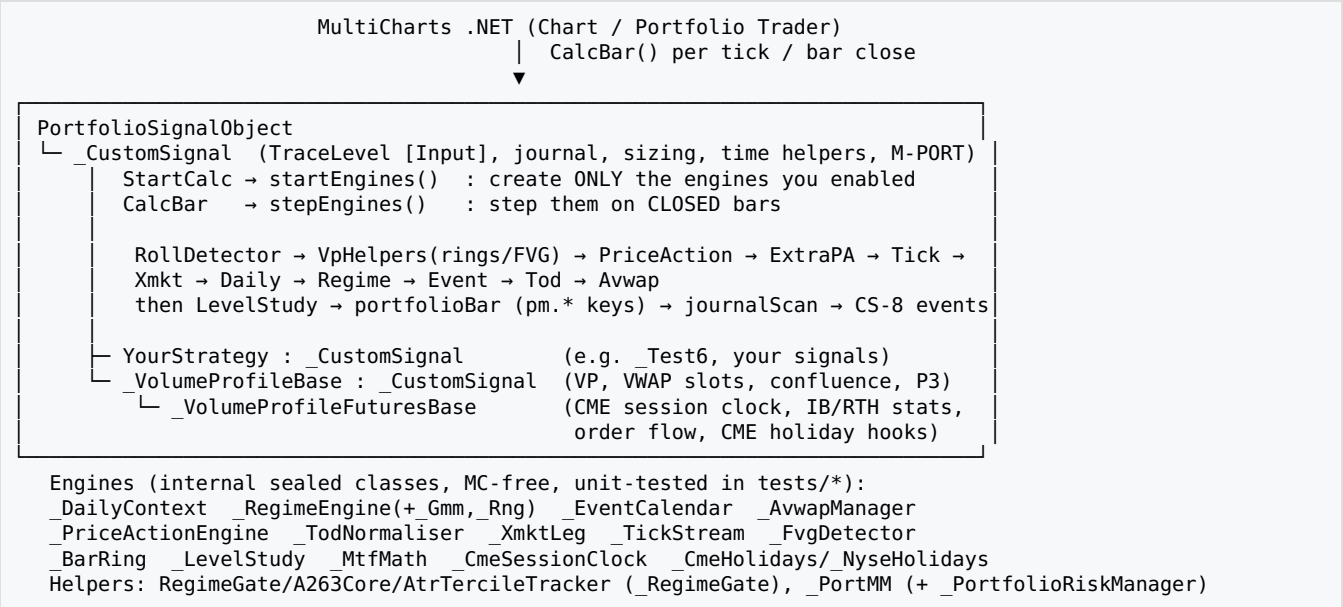
# MultiCharts .NET Engines — User Manual

*Source: `origin/main` at commit `327dafd`, which includes merged PRs #18–#26. Every member name below was checked with `rg` against that tree. Anything I could not confirm from the code is marked **VERIFY**.*

---

# Part 1 — Quick start

## 1.1 Architecture




Some rules hold for every engine:

1. **Lazy.** An engine is created in `StartCalc` only if you enabled it through its **protected config** (never an `[Input]`). If you didn't, its accessor returns `null` (or a neutral state) and logs a one-time Warning.
2. **Closed bars only.** Engines commit on `Bars.Status == EBarState.Close`. An intrabar tick never changes their state.
3. **As-of discipline.** Daily values describe *completed prior sessions*. `RegimeState.AsOf` is the day whose close produced the value, and that value is usable from the **next** session. AVWAP anchors have a `UsableFrom` time. HTF reads use `HtfClosedIndex` / `htfMapped`.
4. **Journaling.** Engines write facts through `engineSink(ETraceLevel.Journal)`, which is `null` when filtered out, so they format no strings. The base never blocks orders on your behalf. For example, `EventBlackout` is only *exposed*.
5. **Same assembly.** The engine types and accessors are `internal`, which works because MC compiles every study into one assembly. Use them as locals (`_DailyContext d = DailyContext;`). Don't expose them in public or protected signatures of a public class.

## 1.2 Minimal `_CustomSignal` skeleton

```
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _MySignal : _CustomSignal
    {
        private const string EntryName = "MyLE", ExitName = "MyLX";
        private IOrderPriced m_le;
        private IOrderMarket m_lx;

        public _MySignal(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();                          // sets TraceLevel = None, NYSE holiday function
            m_le = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, EntryName,
EOrderAction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, ExitName, EOrderActio
n.Sell));
```

```
            // engine configs may be set here (pattern A) ...
        }

        protected override void StartCalc()
        {
            // ... or here, BEFORE base.StartCalc() (pattern B) — engines are created inside
base.StartCalc()
            base.StartCalc();
        }

        protected override void CalcBar()
        {
            base.CalcBar();                      // steps all enabled engines first
            if (Bars.Status != EBarState.Close) return;
            // read engines (values as of this closed bar), decide, Send() orders for the next bar
        }
    }
}
```

`TraceLevel` is the only `[Input]` that `_CustomSignal` defines (`[Input] public virtual ETraceLevel TraceLevel`). Set it to **Journal** in Format Signals. **VERIFY:** `_CustomSignal.Create()` sets `TraceLevel = ETraceLevel.None`, so a code default must be assigned *after* `base.Create()`. Whether the value saved in the dialog then overrides it is MC behaviour I have not confirmed in this repo.

## 1.3 Enabling an engine lazily and setting its config

There are two real patterns in the repo:

```
// Pattern A — in Create() (as _EventCalendarTest does)
protected override void Create()
{
    base.Create();
    EventConfig.enabled = true;
    EventConfig.minTier = EEventTier.T3;
    RollDetection = ERollDetection.Series;
}

// Pattern B — in StartCalc() before base.StartCalc() (as _Test6 / _MtfProbe do)
protected override void StartCalc()
{
    RegimeGate.ApplyArm(RegimeGateConfig, RegimeConfig);   // must precede base.StartCalc
    NeedsDaily = true;
    DailyConfig.atrLength = 20;
    HtfStreams = new[] { 2 };
    base.StartCalc();
}
```

| Engine | Switch (protected, not an Input) | Config object | Accessor |
|---|---|---|---|
| M-DAILY | `NeedsDaily = true` | `DailyConfig` | `DailyContext`, `DailyGapPct` |
| G-R Regime | `RegimeConfig.enabled = true` (forces Daily + HV{20,63}) | `RegimeConfig` | `Regime` (RegimeState copy) |
| M-EVENT | `EventConfig.enabled = true` | `EventConfig` | `EventCalendar`, `EventBlackout`, `EventFlattenNow` |
| M-AVWAP | `AvwapEvents = EAvwapAnchor.…` (≠ None) | `AvwapLimits` | `AvwapManager` |
| PRICEACTION | `PriceActionConfig.enabled = true` / `ExtraPriceAction.Add(...)` | `PriceActionConfig` | `PriceActionEngine`, `PriceActionFor(ds, role)` |
| M-TOD | `TodMetrics = ETodMetric.…` | `TodSessions`, `TodMinSessions`, `TodBucketMinutes` | `TodNormaliser` |
| M-XMKT | `XmktLegs.Add(new XmktLegConfig{…})` | `XmktLegConfig` | `XmktLeg(role)`, `XmktLeaderSwing(role)` |
| M-TICK | `TickConfig.enabled = true` | `TickConfig` | `TickStats` |
| M-MTF | `HtfStreams = new[]{2}` | — | |

| Engine | Switch (protected, not an Input) | Config object | Accessor |
|---|---|---|---|
| | | | `HtfClosedIndex`, `HtfClosed`, `HtfClosedBar`, `htfMapped`, `HtfForming` |
| Rolls | `RollDetection = ERollDetection.Series` | `RollGapTicks`, `RollDates` | `rollOn(ds)`, `RollBars`, `IsRollWindow(day)` |
| FVG | `Fvgs.enabled = true` | `Fvgs` (FvgConfig) | `FvgDetector`, `fvgLevel(f, edge)` |
| Bar rings | `requireRing(ds, bars, consumer)` (after base.StartCalc) | `BarRings` | `Ring(ds)`, `ringBarOf(ds, ago)` |
| A2-33 level study | `LevelStudyConfig.enabled = true` | `LevelStudyConfig` | `levelStudyAdd(...)` |
| M-PORT | automatic in Portfolio Trader (`PortfolioPublish`) | virtual `Portfolio*` props | `portfolioArm`, `portfolioStop` |

**Dependencies are enabled for you.** Regime forces `NeedsDaily`. AVWAP `Swing`/`Breakout` anchors force a PriceAction instance. AVWAP `VolumeClimax`, or a P2 placebo on Data1, forces TOD `Volume`. `XmktLegConfig.leaderSwing` creates a PA on the leg.

## 1.4 Reading outputs without lookahead

```
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;            // 1. closed bars only

    _DailyContext d = DailyContext;                        // 2. null if NeedsDaily == false
    DateTime today = tradingDayKey(Bars.Time[0]);
    bool atrOk = d != null && d.Valid && d.PrevDate < today; // 3. t-1: completed PRIOR session
    double atr = atrOk ? d.Atr20d : double.NaN;

    RegimeState r = Regime;                                // 4. usable iff AsOf < today
    bool regimeOk = r.Valid && !r.UsingFallback && r.AsOf < today;

    int i = HtfClosedIndex(2);                             // 5. HTF: last CLOSED bar of Data2
    double htfClose = BarsOfData(2).Close[i];
}
```

Orders sent on bar *t*'s close execute on bar *t+1* (`MarketNextBar`, or a `Stop`/`Limit` working from the next bar). Every value above is known at *t*'s close, so there is no lookahead. Never read `BarsOfData(n).Close[0]` of a forming HTF bar. Use `HtfForming(ds)` only when you mean it (it is journaled once).

## 1.5 Journaling

- Set `TraceLevel = Journal`. That enables Error/Warning/Info, the engines' Journal facts, the per-trade CSV `MCJournal_<runId>.csv`, and CS-8 signal events `MCEvents_<runId>.csv`.
- Guard expensive strings with `if (journalOn()) trace(ETraceLevel.Journal, $"...")`, or use `traceOn(level)`.
- Call `journalArm(entryName, plannedEntry, initialStop, target, context)` next to `Send()` so the trade row carries R and context.
- Call `logSignal(tag, dir, price, stop, context, taken)` to record taken *and* untaken signals. The base tracks forward MFE/MAE in R at `SignalHorizons` (default {5,15,30,60} bars).

```
if (journalOn())
{
    var ctx = new Dictionary<string, string> { { "atr", atr.ToString("0.00") }, { "regime", r.Label.ToStrin
g() } };
    journalArm(EntryName, stopPrice, initialStop, double.NaN, ctx);
    logSignal("ORB", +1, stopPrice, initialStop, ctx, true);
}
m_le.Send(stopPrice, qty);
```

## 1.6 Important for @ES on a plain `_CustomSignal`: the CME session adapter

On a plain `_CustomSignal`, `tradingDayKey()` defaults to the **calendar date** (`ETradingDayRoll.Calendar`). `sessionClock()` defaults to the bar's own time of day, and `isRthBar()` returns **true** for every bar. With a 24-hour Globex @ES chart, that means:

- the Sunday/evening 18:00 ET session is split at midnight;
- M-DAILY aggregates calendar days, not Globex days, and its RTH-close basis becomes "last bar of the day";
- `NoEntryBefore` / `NoEntryAfter` / `FlatBy` are compared in chart time.

`_VolumeProfileFuturesBase` fixes all of this by overriding these virtuals with a `_CmeSessionClock`. A plain `_CustomSignal` can do the same, because every member used here is a real protected virtual or public member:

```
private _CmeSessionClock m_clock;
private _CmeSessionClock Clock
{
    get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBarTimeZone(Bars.Info)); return m_cloc
k; }
}
protected override void StartCalc() { m_clock = null; /* engine configs … */ base.StartCalc(); }

protected override DateTime tradingDayKey(DateTime barTime) { return Clock.GlobexDate(barTime); }
protected override TimeSpan sessionClock(DateTime barTime) { return Clock.ToEastern(barTime).TimeOfDay; }
protected override bool isRthBar(DateTime barTime) { return Clock.IsRthValueBarEt(Clock.ToEastern(barTime))
; }
protected override int todMinuteOfSession(DateTime barTime)
{
    int m = (int)Clock.ToEastern(barTime).TimeOfDay.TotalMinutes - (int)Clock.GlobexOpen.TotalMinutes;
    return m < 0 ? m + 1440 : m;
}
protected override bool todHalfDay(DateTime d)
{
    return Clock.IsNoRthDay(d) || Clock.HolidayHaltEt(d) != null || Clock.RthEarlyCloseEt(d) != null;
}
```

These bodies mirror `_VolumeProfileFuturesBase` lines 356–512. **VERIFY:** `_Test6` does *not* override `tradingDayKey`. With a Globex session template, its M-DAILY days are therefore calendar days. Decide whether that is intended (see the pitfalls in 1.8). Also, on a plain `_CustomSignal`, `PriceActionConfig` keeps the **Library** preset. Call `PriceActionConfig.ApplyPreset(EPriceActionPreset.Futures)` in `Create()` to match the futures bases (LeftInclusive ties, `volumeIncludesCurrent = false`).

## 1.7 Worked example: session clock + daily ATR + regime gate + event blackout + AVWAP filter

This is a long-only opening-range breakout:

- **Entry:** a buy stop at RTH open + k·ATR20d, latched on the first bar closing after 09:30 ET.
- **Filters** (all as of t-1 or t):
- the HMM regime of the prior session is not label 2;
- no T1 macro blackout (±10 min);
- the RTH open is above the week-open AVWAP.
- **Exit:** flat at 15:55 ET, or 5 min before a CME early close.
- **Skips:** no trade on no-RTH days.

```
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _EsOrbExample : _CustomSignal
    {
        private const string EntryName = "OrbLE", ExitName = "OrbLX";
        private static readonly TimeSpan RthOpen = new TimeSpan(9, 30, 0), LastEntry = new TimeSpan(15,
```

```
30, 0), Flat = new TimeSpan(15, 55, 0);
        private const double K = 0.5;

        // strategy settings in a protected config struct, not Inputs
        protected struct OrbConfig { public double k; public int contracts; public int blackoutMin; }
        protected OrbConfig Cfg = new OrbConfig { k = K, contracts = 1, blackoutMin = 10 };

        private IOrderPriced m_le;
        private IOrderMarket m_lx;
        private _CmeSessionClock m_clock;
        private DateTime m_latchedDay = DateTime.MinValue;
        private double m_stop = double.NaN;
        private TimeSpan m_flatEt;

        public _EsOrbExample(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock
        {
            get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBarTimeZone(Bars.Info)); retur
n m_clock; }
        }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            m_le = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, EntryName,
EOrderAction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, ExitName, EOrderActio
n.Sell));
        }

        protected override void StartCalc()
        {
            m_clock = null;
            RollDetection = ERollDetection.Series;          // continuous unadjusted @ES: detect splices
            NeedsDaily = true;  DailyConfig.atrLength = 20; // M-DAILY Wilder ATR20d
            RegimeConfig.enabled = true;                    // G-R (forces Daily + HV20/63)
            EventConfig.enabled = true;                     // M-EVENT
            EventConfig.blackoutTier = EEventTier.T1;
            EventConfig.blackoutPreMin = Cfg.blackoutMin;
            EventConfig.blackoutPostMin = Cfg.blackoutMin;
            AvwapEvents = EAvwapAnchor.WeekOpen;            // M-AVWAP week-open anchor on Data1
            MaxTradesPerDay = 1;
            base.StartCalc();                               // engines are created here
            m_latchedDay = DateTime.MinValue; m_stop = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;

            DateTime et = Clock.ToEastern(Bars.Time[0]);
            DateTime day = tradingDayKey(Bars.Time[0]);
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);

            // exit: 15:55 ET, or early close - 5 min
            if (StrategyInfo.MarketPosition > 0 && et.TimeOfDay >= m_flatEt) { m_lx.Send(); return; }

            // latch on the first bar closing after 09:30 ET
            if (day != m_latchedDay && et.TimeOfDay > RthOpen && !cme.NoRth)
            {
                m_latchedDay = day;
                m_flatEt = cme.RthCloseEt.HasValue ? cme.RthCloseEt.Value - TimeSpan.FromMinutes(5) : Flat;

                _DailyContext d = DailyContext;
                bool atrOk = d != null && d.Valid && d.PrevDate < day;
                RegimeState r = Regime;
                bool regimeOk = r.Valid && !r.UsingFallback && r.AsOf < day && r.Label != 2;
                _AvwapManager am = AvwapManager;
                _Avwap wk = am != null ? am.Latest(EAvwapAnchor.WeekOpen) : null;
                bool avOk = wk != null && wk.Valid && wk.UsableFrom <= Bars.Time[0] && Bars.Open[0] > wk.Va
lue;

                m_stop = atrOk && regimeOk && avOk ? Bars.Open[0] + Cfg.k * d.Atr20d : double.NaN;
```

```
                if (journalOn())
                    trace(ETraceLevel.Journal, $"ORB {day:yyyy-MM-dd} O({Bars.Open[0]}) atr({(atrOk ?
d.Atr20d : double.NaN):0.00}) " +
                          $"regime({r.Label}) asOf({r.AsOf:yyyy-MM-dd}) avwapWk({(wk != null ? wk.Value :
double.NaN):0.00}) stop({m_stop:0.00}) " +
                          $"halfDay({cme.RthCloseEt.HasValue}) noRth({cme.NoRth})");
            }

            // working window: from the bar after the latch until 15:30 ET, flat, no blackout
            string why;
            if (day == m_latchedDay && !double.IsNaN(m_stop) && StrategyInfo.MarketPosition == 0
                && et.TimeOfDay < LastEntry && !EventBlackout && entryAllowed(out why))
            {
                if (journalOn())
                {
                    var ctx = new Dictionary<string, string> { { "regime", Regime.Label.ToString() } };
                    journalArm(EntryName, m_stop, double.NaN, double.NaN, ctx);
                }
                m_le.Send(m_stop, Cfg.contracts);
            }
        }
    }
}
```

Notes on the example:

- It is compiled from real members. The strategy logic itself is illustrative and not pre-registered.
- `EventBlackout` reflects the **just-closed bar** `[open, close)`. The stop works on the *next* bar. To also block a next-bar release, check `EventCalendar.MinutesTo(EEventTier.T1)` against `blackoutPreMin + bar minutes`. **VERIFY** the window semantics you want.
- The M-DAILY ATR is **roll-gap adjusted** because `RollDetection` is on (`DAILY roll reset … ATR reseeding`). With `RollDetection = Off`, the roll gap enters the ATR, and on a futures symbol the base warns once.
- The Regime **USE RULE** (from `_CustomSignal` comments): regime is a prior/bucket, never a trigger. A consumer must be pre-registered against the AtrTercile baseline before it is used. `_Test6` / `RegimeGate` is the pre-registered form.

## 1.8 Pitfalls checklist

- [ ] **Lookahead.**
- Read engines only after `base.CalcBar()` and on `EBarState.Close`.
- Daily values require `PrevDate < today`; Regime requires `AsOf < today`; AVWAP requires `UsableFrom <= Bars.Time[0]`; HTF requires `HtfClosedIndex` / `htfMapped`, never `[0]` of a forming bar.
- TOD norms use prior completed sessions only.
- [ ] **Rolls (continuous unadjusted @ES).**
- Set `RollDetection = ERollDetection.Series` (or `Table` + `RollDates`). It is **Off** by default, and then nothing is roll-adjusted.
- `rollOn(1)` is true on the first bar of the new contract. Engines reset or adjust there: Daily `OnRoll`, PA `roll reset`, AVWAP `invalidated (roll)`, and a Regime refit deferred on the roll day.
- Exclude roll ±1 via `IsRollWindow(day)`. This works only when rings/FVG or RollDetection are active.
- [ ] **Half-days and holidays.**
- Use `_CmeHolidays.Rules.Get(globexDate)`: `.Closed`, `.NoRth`, `.HaltEt`, `.RthCloseEt`.
- TOD excludes half-days only if `todHalfDay` is overridden (the futures base does this, a plain `_CustomSignal` returns false).
- The Event calendar flags `EventDay.isHalfDay`.
- [ ] **Trading day.** On a plain `_CustomSignal` with a Globex template, override `tradingDayKey` / `sessionClock` / `isRthBar` (see 1.6).
- [ ] **Multi-data streams.**
- Orders always go to Data1.
- `HtfStreams` must carry the Data1 root; other markets go through `XmktLegs`.
- A leg with another root never inherits Data1 rolls.
- `requireDataStream` throws at StartCalc if a stream is missing.
- A leg is `Available` only if not stale (`maxStaleSec`, or 2× the leg bar).
- [ ] **Portfolio Trader.**

- `PortfolioPublish` is true only in PT.
- `sizeForRisk` arms M-PORT for Data1. Otherwise call `portfolioArm(side, entry, stop, qty)` next to `Send`, and `portfolioStop(stop)` when you trail.
- Apply `_PortfolioRiskManager` as the PT money-management signal. It sizes via `EntryContracts` and never blocks exits.
- `equityAvailable` in PT uses `Portfolio.MaxRiskEquityPerPosPercent`. The *Percent parameters are fractions (0.01 = 1%).
- [ ] **Validation data.**
- Engines step on closed Data1 bars, so the bar size changes results (`_Test6` journals a bar-size deviation).
- Validate on 1-min with Bar Magnifier, or on tick (`TickConfig` uses a DataLoader 1-tick request; its size outputs stay parked until `sizeOutputsVerified`).
- [ ] **Config timing.** Set configs in `Create()` or before `base.StartCalc()`. After that, the engines are already built. The exception is roll settings, which are read live.
- [ ] **Event table coverage.** If a table kind starts after your first bar, the Event engine **disables itself** with an Error line unless `allowPartialCoverage = true`.

---

# Part 2 — Engines and helpers

> Each chapter covers the same items: **purpose · enable · config defaults · API · timing · journal · tests · snippet**.

## 2.1 `_CustomSignal` core services

- **Purpose:** the base for every strategy. It provides tracing, the journals, sizing, trade-management rules, time-zone helpers, the trading-day key, the RUN/RUN-END lines and M-PORT publishing.
- **Enable:** always on. The only `[Input]` is `TraceLevel`.
- **API (real signatures):**
- Tracing: `traceOn(ETraceLevel)`, `trace(ETraceLevel, string)`, `journalOn()`, `engineSink(ETraceLevel)`.
- Journaling: `journalArm(string entryOrderName, double plannedEntry, double initialStop, double target, IDictionary<string,string> context = null)`, `logSignal(string tag, int direction, double price, double stop, IDictionary<string,string> context, bool taken)`, `SignalHorizons` (default `{5,15,30,60}`).
- Trade management (CS-7, properties, not Inputs): `NoEntryBefore`, `NoEntryAfter`, `FlatBy` (`TimeSpan?`, compared with `sessionClock()`), `MaxTradesPerDay`, `DailyLossStopCount`, `DailyLossStopR`, `RiskUnitDollars`, `TimeStopBars`; `entryAllowed(out string reason)`, `mustFlatten()`, `TradesToday`, `LosersToday`, `NetProfitToday`.
- Sizing (CS-4): `sizeForRisk(IInstrument bars, double entry, double stop, double riskFraction, SizingLimits limits)`, where `SizingLimits` has `maxQty`, `maxNotionalFraction`, `unitNotional`, `minStopTicks`, `roundLots`. Also `equityAvailable(double)` and `costFilterPasses(IInstrument, entry, stop, out reason)` with `CostFilterMaxFractionOfR` (default 0 = off).
- Time (CS-6): `ToZone(DateTime barTime, string tzId)`, `inTimeWindow(TimeSpan start, TimeSpan end, string tzId)`, `BarTimeZone`, `resolveBarTimeZone(IInstrumentSettings)`, `TradingDayRoll` (`Calendar` default, or `Auto`), `tradingDayKey(DateTime)` virtual, `sessionClock(DateTime)` virtual, `Holidays` (`_NyseHolidays`).
- **Timing:** the CS-7 counters use `tradingDayKey`. `mustFlatten` is true at or after `FlatBy`, or once `TimeStopBars` have elapsed.
- **Journal:**
- `RUN id(..) build(..) strategy(..) symbol(..) category(..) resolution(..) timeZone(..) firstBar(..) barNumber(..) app(..) inputs[..]`, plus `event[canon] journalSchema(CS-1+evtDayTier,evtMinSince)` when Event is on.
- `RUN-END id(..) lastBar(..) barNumber(..) realTime(..)`.
- `entry blocked: <reason>` at Info.
- **Snippet:**

```
NoEntryBefore = new TimeSpan(9, 35, 0); NoEntryAfter = new TimeSpan(15, 30, 0); FlatBy = new TimeSpan(15, 55, 0);
MaxTradesPerDay = 2; DailyLossStopCount = 2;
// CalcBar:
if (mustFlatten()) m_lx.Send();
string why;
if (StrategyInfo.MarketPosition == 0 && entryAllowed(out why))
{
    int qty = sizeForRisk(Bars, entry, stop, 0.005, new SizingLimits { maxQty = 5, minStopTicks = 4 });
    if (qty > 0) m_le.Send(entry, qty);
}
```

## 2.2 `_CmeSessionClock` and the CME/NYSE calendars

- **Purpose:** CME Globex session windows in US Eastern time (Asia, Europe, NewYork, Rth, Premarket, PostMarket), the Globex trading date, halt/closed state, and holiday hooks.
- **Enable:** `new _CmeSessionClock(resolveBarTimeZone(Bars.Info))` (as in `_CmeSessionClockTest`). On the futures bases it is built in and exposed as `SessionClock`.
- **Defaults (ET):**
- Globex open 18:00, close 17:00; halt 17:00–18:00.
- Asia 18:00–03:00; Europe 03:00–11:30; NewYork 08:00–17:00.
- `RthStart` 09:30, `RthEnd` 16:15 (session flag), `RthValueEnd` 16:00 (value window).
- `UseCmeHolidayTable = true`.

- **API:**
- Conversion and state: `ToEastern`, `EasternOf`, `State(barTime)` → `GlobexState {Closed, Halt, Open}`, `Sessions(barTime)` → `[Flags] MarketSession`, `InSession`, `InAny`, `GlobexDate(barTime)`, `GlobexDateEt`, `PrimarySessionEt`.
- Session timing: `TimeSinceOpenEt` / `MinutesSinceOpenEt(et, session)`, `TimeUntilCloseEt` / `MinutesUntilCloseEt`, `SessionLength`, `IsOpeningRangeEt(et, session, minutes)`, `IsRthAfterInitialBalanceEt(et, ibMinutes)`, `TrySessionBoundsEt(et, session, out open, out close)`, `TryWindow`.
- RTH value window: `RthValueEndEt(globexDate)`, `IsRthValueBarEt(et)` (close-stamped bar in `(RthStart, RthValueEnd]`).
- Holidays: `IsGlobexClosedDay`, `IsNoRthDay` (NYSE holiday that is not a Globex closure), `HolidayHaltEt`, `RthEarlyCloseEt`. Hooks: `GlobexClosedDayHook`, `NoRthDayHook`, `HolidayHaltHook`, `RthEarlyCloseHook`.
- Static: `DefaultGlobexClosedDay`, `InWindow`, `IsSingleFlag`.
- **`_CmeHolidays.Rules`:**
- `Get(date)` returns a `Day` with `Closed`, `NoRth`, `HaltEt`, `RthCloseEt` and `Name`.
- Constants: `HolidayHaltEt` 13:00, `EarlyCloseEt` 13:15, `GoodFridayCloseEt` 09:15, `MourningCloseEt` 09:30.
- Helpers: `ClosedDates(year)`, `Observed`, `NthWeekday`, `LastWeekday`, `Easter`.
- **Futures-base wrappers** (`_VolumeProfileFuturesBase`): `IsRth()`, `IsAsia()`, `IsEurope()`, `IsNewYork()`, `IsLondonNyOverlap()`, `GetGlobexState()`, `GetSessions()`, `MinutesSinceOpen(session)`, `MinutesUntilClose(session)`, `IsOpeningRange(session, minutes)`, `IsRthAfterInitialBalance(ib)`, `TrySessionBounds(...)`, `IsNewGlobexDay()`, `Session` (SessionStats: RTH/ETH/IB, prior-day H/L/C, `IbRange`, `Gap`), `IbMinutes` (60).
- **Timing:** bars are close-stamped. A bar is RTH when its ET close is in `(09:30, value end]`.
- **Journal:** the test signal prints `VPF-7 clock globexDate(..) firstBarEt(..) state(..) sessions(..) noRth(..) rth(..)`.
- **Tests:** `tests/SessionClockTests`, `tests/CalendarTests` (with `gen.sh`).
- **Snippet:**

```
DateTime et = m_clock.EasternOf(Bars.Time[0]);
DateTime gx = m_clock.GlobexDate(Bars.Time[0]);
DateTime o, c;
if (m_clock.TrySessionBoundsEt(et, _CmeSessionClock.MarketSession.Rth, out o, out c) && !
m_clock.IsNoRthDay(gx))
{
    int mins = m_clock.MinutesSinceOpenEt(et, _CmeSessionClock.MarketSession.Rth);
}
```

### 2.3 M-DAILY `_DailyContext`

- **Purpose:** daily/weekly context: Wilder ATR, SMA200, RSI2, TSMOM, HV and HV percentile, LBR, prior day and week levels, and the gap. Roll gaps are removed.
- **Enable:** `NeedsDaily = true`. Regime enables it automatically.
- **`DailyConfig` defaults:**
- `dataStream 0` (aggregate Data1 per `tradingDayKey`), `weeklyStream 0`;
- `atrLength 20`, `smaLength 200`, `rsiLength 2`, `tsmomLookbacks {252,126,63}`;
- `hvLengths {}`, `hvAnnualisation 252`, `hvPctLookback 0`, `lbrRsiLength 0`;
- `closeBasis EDailyClose.RthValue`; `HvOn`; `Validate()`.
- **API:** `Atr20d` (ATR of length `atrLength`, despite the name), `Sma200`, `Rsi2`, `Tsmom(n)`, `Hv(n)`, `HvPercentile(n)`, `Roc1`, `Lbr`, `RthCloseAdj`, `RthClosePrev`, `PrevHigh` / `PrevLow` / `PrevClose` / `PrevDate` / `PrevRange`, `PrevWeekHigh` / `PrevWeekLow`, `Valid`, `Days`, `GapPct(todayOpen)`, and the host's `DailyGapPct`.
- **Timing:** a day is added when the *next* trading day's first bar arrives. On that bar, `PrevDate` is yesterday, so every value describes completed prior sessions. With a daily data stream, only the closed daily bar (`HtfClosedIndex`) whose day is before today is used. Rolls apply `OnRoll(day, rollBarOpen - prevClose)`.
- **Journal:**
- `DAILY {date} return spans a gap of N business days since {prev}`;
- `DAILY hv{n} first valid {date} …`;
- `DAILY hv{n}pct first valid …`;
- `DAILY roll reset {date}: history adjusted by {d}, ATR reseeding`;
- `DAILY closeBasis=Bar (P3 unverified): …`;

- a Warning: `DAILY Data{n} advanced k bars between Data1 bars`.
- **Tests:** `tests/DailyTests` (`gen.sh` generates the calendars).
- **Snippet:**

```
NeedsDaily = true; DailyConfig.hvLengths = new[] { 20 };      // before base.StartCalc()
// CalcBar
_DailyContext d = DailyContext;
if (d != null && d.Valid && d.PrevDate < tradingDayKey(Bars.Time[0]))
    if (Bars.Close[0] > d.PrevHigh && d.Hv(20) < 0.25) { /* … */ }
```

## 2.4 G-R regime engine `_RegimeEngine`

- **Purpose:** an HMM (or GMM) market regime on daily features, with hysteresis and an ATR-tercile fallback.
- **Enable:** `RegimeConfig.enabled = true`. This forces M-DAILY with HV {20, 63}.
- **`RegimeConfig` defaults:**
- Model: `method Hmm`, `K 3`, `kSelect Fixed`, `features {Return, LogHv20, Tsmom63z}`.
- Fitting: `fitSessions 1000`, `minSessions 500`, `refit Monthly`, `refitEveryN 21`, `restarts 10`, `maxIter 200`, `tol 1e-6`, `regCovar 1e-6`, `transPrior 1`.
- Filtering and switching: `warmFilter 63`, `switchProb 0.60`, `switchMargin 0.10`.
- Other: `seed 20261021`, `fallback AtrTercile`, `publish false`, `atrPctLookback 250`, `parkinsonBars 10`, `bicDiagnostic true`, `rollSuspectZ 5`, `gapBusinessDays 4`.
- Methods: `Validate()`, `Copy()`, `IndexOf`.
- **Enums:** `ERegimeMethod {Hmm, Gmm}`, `ERegimeKSelect {Fixed, Bic}`, `ERegimeRefit {Monthly, EveryNSessions, Quarterly}`, `ERegimeFallback {AtrTercile, Unknown}`, `ERegimeFeature {Return, LogHv20, Tsmom63z, Parkinson10}`.
- **API:** the `Regime` property returns a copy of `RegimeState`:
- fields: `Valid`, `Label` (0 = calmest, -1 invalid), `RawArgmax`, `Probs`, `AsOf`, `FittedThrough`, `K`, `LogLik`, `Refits`, `UsingFallback`, `RelabelConflicts`;
- methods: `Copy()`, `Neutral(K)`.
- When the engine is disabled, `Regime` returns a neutral state (`Valid=false`, `Label=-1`).
- Engine statics: `AtrPercentile`, `AtrBucket`, `Hysteresis`, `Argmax`.
- **Timing:** the engine is fed once per completed M-DAILY day. `AsOf` is that day, and the value is usable from the **next** session. `FittedThrough < AsOf`. Refits on a roll day are deferred.
- **M-PORT:** with `publish = true`, `pm.regime = Label+1` and `pm.regimeP = Probs` are published at session close.
- **Journal:**
- `REGIME {date} label() raw() p() fb();`
- `REGIME switch a->b {date};`
- `REGIME refit {date} window() K() ll() iters() conv() means();`
- `REGIME diag {date} bicK…;`
- `REGIME fallback reason(…);`
- `REGIME refit deferred {date}: roll day;`
- `REGIME gap …;`
- `REGIME roll adjust suspect …;`
- `REGIME relabel-conflict ….`
- **Tests:** `tests/RegimeTests` (`reference.py`; `run.sh` can `--export gv`), `tests/GmmTests`.
- **Snippet:**

```
RegimeConfig.enabled = true;                    // before base.StartCalc()
// CalcBar
RegimeState r = Regime;
bool usable = r.Valid && !r.UsingFallback && r.AsOf < tradingDayKey(Bars.Time[0]);
int bucket = usable ? r.Label : -1;             // a PRIOR / bucket, never a trigger
```

## 2.5 RegimeGate / A2-63 helpers (`_RegimeGate.Function.CS`)

- **Purpose:** the pre-registered regime consumer used by `_Test6`: the U/G/A arms, the decision, canonical hashing and the A2-63 host core.

- **Enable:** construct the objects yourself, then call `RegimeGate.ApplyArm(RegimeGateConfig, RegimeConfig)` before `base.StartCalc()`.
- **Defaults:**
- `RegimeGateConfig { arm = ERegimeGateArm.U, placeboSeed = RegimeGate.PlaceboSeed (20261022) }`.
- `A263Config { k 0.5, atrLength 20, rthOpen 09:30, activeEnd 15:30, exitTime 15:55, rthClose 16:00, contracts 1, halfDayExitBefore 5m }`.
- **API:**
- `RegimeGate.Decide(arm, RegimeState, session, atrTercile)` returns a `RegimeGateDecision` with `session`, `asOf`, `arm`, `label`, `raw`, `atrTercile`, `pFilt`, `fb`, `usable`, `wouldBlock`, `allow` and `reason` (an `ERegimeGateReason {Ok, Warmup, Invalid, Fallback}`).
- `RegimeGate.Line`, `TradeFields`, `HashFields`, `Canon`, `Hash`, `Fnv1a64`; constants `BlockedLabel = 2` and `Version = "A263-RG-2"`.
- `AtrTercileTracker(lookback, K)`: `.AddDay(date, atr)`, `.Tercile`, `.LastDate`.
- `A263Core(cfg)`: `IsLatchBar(etClose)`, `Latch(etClose, open, atr, atrAsOf, allow, earlyCloseEt, noRth)`, `Step(etClose, isLong)` returning `A263Action {latched, sendStop, stop, exit}`, plus `HasSignal`, `Threshold`, `Suffix()`.
- `RegimeGateCounts.Add(d, noRth)`, `.Suffix()`.
- **Arms:** U = ungated; G = skip when the prior session's HMM label == 2; A = skip ATR tercile 2 (via `fitSessions = minSessions = 100000`).
- **Hash pins:** U `26b38655261cc0bf`, G `9c1da7b123ca9441`, A `3d9814a5f264c353`.
- **Journal:**
- Header: `REGGATE header version() arm() cfg() seed() symbol() bar() barDeviation() firstBar() canon[]`.
- Per session: `REGGATE {date} arm() asOf() label() raw() pFilt() fb() usable() allow() atrT() wouldBlock() why() signal() cfg() seed()`, followed by `O() atr20d() atrAsOf() atrOk() thr() exitAt() halfDay() noRth()` and the counts `nSess() nUsable() nWarm() nInv() nFb() nBlock() nNoRth()`.
- **Tests:** `tests/RegimeGateTests` (`gen.sh`).
- **Snippet:** see `_Test6.CalcBar` (it is reproduced in essence in 1.7).

### 2.6 M-EVENT `_EventCalendar`

- **Purpose:** a scheduled-event calendar with as-of visibility. It covers:
- macro releases: FOMC, CPI, PPI, NFP, JOLTS, Claims, PCE, GDP, Retail, ISM, Jackson Hole, testimony;
- energy: EIA;
- calendar events: quarterly expiry, roll week, monthly opex, index rebalance, half-day, VIX expiry, Russell recon.
- **Enable:** `EventConfig.enabled = true`.
- **`EventConfig` defaults:**
- Scope: `classes Macro|Calendar`, `minTier T3`, `tierOverride {}`, `csvPath ""`, `allowPartialCoverage false`.
- Blackout: `blackoutPreMin 0`, `blackoutPostMin 0` (0/0 = no blackout), `blackoutTier T1`, `blackoutFlatten false`.
- Other: `publish false`, `includeRussell false`; methods `Validate()`, `Canon()`.
- **Enums:**
- `EEventTier {None=0, T3=1, T2=2, T1=3, C=8}` (T1 is the highest);
- `EEventClassMask {None, Macro=1, Calendar=2, Energy=4, All=7}`;
- `EEventKind` (FomcDecision=1 ... EiaPetroleum=18, QuarterlyExpiry=30 ... RussellRecon=36, Custom=62);
- `EEventFlags` (Sep, Quad, Unscheduled, Scheduled, Delayed, Canceled, Csv, RuleOnly, Pm).
- **Records:**
- `EventRec` fields: `kind`, `tier`, `flags`, `etTime`, `etEnd`, `utc`, `tradingDay`, `knownSinceUtc`, `canceledSinceUtc`, `refPeriod`, `name`, `source`; plus `IsSpan`, `Class`, `KindName`.
- `EventDay` fields: `day`, `maxTier`, `kindsMask`, `hasFomc`, `hasSep`, `isHalfDay`, `isRollWeek`, `isQuarterlyExpiry`, `isOpex`, `isRebalance`, `firstReleaseEt` (TimeSpan?), `count`, `eiaRuleOnly`; plus `Has(kind)`.
- **API:**
- State: `TradingDay`, `EventBlackout`, `EventFlattenNow`, `NowUtc`, `Disabled`.
- Next and previous events: `NextEvent(minTier, out e)`, `PrevEvent(minTier, out e)`, and overloads `(nowUtc, minTier, mask, out e)`.

- Distance in minutes: `MinutesTo(minTier, mask = Macro)`, `MinutesSince`, `MinutesSinceAt`.
- Windows: `IsInWindow(minTier, pre, post, mask = Macro)`, `IsInWindowAt`.
- Day queries: `EventsOn(day, mask = All)`, `EventsOnExPost`, `EventsInRange`, `Classify(day)`, `Classify(day, asOfUtc)`, `ClassifyExPost(day)`.
- Expiry and rolls: `IsRollWeek()`, `IsRollDay()`, `SessionsToExpiry()`, `InSpan(kind)`, `LastT1Instant(out et)`.
- Conversion: `ToEt`, `TradingDayOf`, `SessionOpenUtc`.
- Statics: `RuleRollDate`, `RollDateFor`, and `EventRules` (`IsNyseBusinessDay`, `ThirdFriday`, `OpexDate`, `RollDate`, `VixExpiry`, `TableSha`).
- Host wrappers: `EventBlackout`, `EventFlattenNow` (false when disabled).
- **Timing:**
- Stepped once per closed bar with `[open, close)` in UTC.
- An event is visible only once it is known (`knownSinceUtc <= now`) or has happened. Cancellations are honoured as of now.
- `Classify(day)` is as of the session open.
- The blackout uses the Macro class only, so Energy consumers call `IsInWindow(..., EEventClassMask.Energy)` explicitly.
- Window queries on daily bars return false.
- **The base never blocks orders.** It is your job to act on `EventBlackout` / `EventFlattenNow`.
- **M-PORT keys** (`publish = true`): `pm.evtTier`, `pm.evtMask` (daily); `pm.evtWin`, `pm.evtNext` (per bar).
- **CS-1 extra columns:** `evtDayTier`, `evtMinSince`.
- **Journal:**
- `EVENT created classes() minTier() blackout(pre/post tier) …;`
- `EVENT table missing: …;`
- `EVENT coverage rules(…) …;`
- `EVENT correction …;`
- `EVENT day {yyyy-MM-dd} at {bar} tier() kinds(Kind@HH:mm,…) firstRelease() [rollWeek] [halfDay] [EiaRuleOnly];`
- `EVENT blackout on|off at {bar} {Kind} {et} ET pre() post() [flatten];`
- an Error line ending `…; engine disabled` when coverage fails.
- **Tests:** `tests/EventTests` (`gen.sh`). Tooling lives in `tools/m-event`.
- **Snippet:**

```
EventConfig.enabled = true; EventConfig.blackoutPreMin = 5; EventConfig.blackoutPostMin = 15;
EventConfig.blackoutFlatten = true;
// CalcBar
if (EventFlattenNow && StrategyInfo.MarketPosition != 0) m_lx.Send();
_EventCalendar ev = EventCalendar;
if (ev != null)
{
    EventDay today = ev.Classify(ev.TradingDay);
    bool fomcDay = today.hasFomc;
    double toT1 = ev.MinutesTo(EEventTier.T1);
    EventRec next;
    if (ev.NextEvent(EEventTier.T2, out next) && journalOn()) trace(ETraceLevel.Journal, $"next
{next.KindName} {next.etTime:HH:mm}");
}
```

## 2.7 M-AVWAP `_AvwapManager`

- **Purpose:** anchored VWAPs on events, with σ-bands, slope, age, touches and expiry. Optional placebo twins (P1/P2) are available.
- **Enable:** `AvwapEvents = EAvwapAnchor.WeekOpen | EAvwapAnchor.GapOpen | …`.
- **`EAvwapAnchor` flags:** `SwingHigh=1`, `SwingLow=2`, `Breakout=4`, `GapOpen=8`, `VolumeClimax=16`, `WeekOpen=32`, `MonthOpen=64`, `QuarterOpen=128`, `NewsBar=256` (inert), and `Swing`.
- **`AvwapLimits` defaults:**
- Anchors: `maxAnchors 6`, `maxAgeSessions 10`, `replaceSameType true`.
- Bands and slope: `bandSigmas {1,2,3}`, `slopeBars 5`, `dataStream 1`.
- Detection thresholds: `gapAtrFraction 0.5`, `gapFallbackTrMultiple 3`, `climaxZ 3`, `fallbackLength 20`, `backfillBars 2`.
- Placebos: `placebos EAvwapPlacebo.None`, `p2MinPool 10`.

- **API:**
- Manager: `Anchors` (`IReadOnlyList<_Avwap>`), `Latest(EAvwapAnchor type)`, `LastBar`, `FallbackTrAvg`.
- Each `_Avwap` has:
    - identity: `Id`, `Type`, `AnchorTime`, `AnchorBar`, `UsableFrom`, `Direction`;
    - value: `Value`, `Sigma`, `SlopeTicks`, `SlopeAtr`, `Band(k)`, `Valid`;
    - lifecycle: `AgeBars`, `AgeSessions`, `Touched`, `TouchCount`, `Expired`, `ExpireReason`;
    - tags: `Tag` (`AV:{Type}:{yyyyMMddHHmm}`), `TagFor(arm)`;
    - placebos: `P1`, `P1Sigma`, `P2`, `P2Sigma`, `p2Degraded`.
- **Timing:**
- Swing anchors become usable at PA pivot *confirmation* (`UsableFrom`); other anchors at the anchor time.
- GapOpen uses `isRthBar` and M-DAILY ATR when available.
- All anchors are invalidated on a roll.
- **Dependencies:** Swing and Breakout force PriceAction. VolumeClimax and P2 force TOD Volume.
- **Journal:**
- `AV created {Tag} usable={t} value={v};`
- `AV expired ({why}) {Tag} at {t};`
- `AV invalidated (roll) {Tag} at {t};`
- `AV p2Degraded …`.
- **Tests:** `tests/AvwapTests`.
- **Snippet:**

```
AvwapEvents = EAvwapAnchor.Swing | EAvwapAnchor.GapOpen;
// CalcBar
_AvwapManager am = AvwapManager;
if (am != null)
    foreach (_Avwap a in am.Anchors)
        if (a.Valid && a.UsableFrom <= Bars.Time[0] && Bars.Low[0] <= a.Band(-1) && a.Direction > 0) { /*
long pullback to -1σ */ }
```

## 2.8 PRICEACTION `_PriceActionEngine`

- **Purpose:** pivots (fractal or ATR zig-zag) on several layers, HH/LH/HL/LL labels, BOS/CHoCH, structure, equal highs/lows, and candle and volume flags.
- **Enable:** `PriceActionConfig.enabled = true`. Extra instances go in `ExtraPriceAction.Add(new PriceActionConfig{ enabled = true, dataStream = 2, role = "htf" })`.
- **`PriceActionConfig` defaults:**
- Stream and pivots: `dataStream 1`, `pivotMode Fractal`, `tieRule Strict`, `layers 2`, `strength 1`.
- ATR and volume: `atrLength 20`, `volumeLength 20`, `volumeIncludesCurrent true`, `atrMultiple 1.5`.
- Candle factors: `factorDoji 0.1`, `factorClose 0.1`, `factorStrong 1.0`, `factorEpsilon 0.05`, `equalFactor 0.1`.
- Breakouts and history: `lookbackPivots 5`, `minBreakoutLayer 1`, `breakoutNeedsStrong true`, `history 50`, `candles true`.
- Presets: `preset Library`, `role ""`; `ApplyPreset(EPriceActionPreset)`, `ApplyPresetDefaults`. The Futures preset uses `LeftInclusive` and `volumeIncludesCurrent = false`, and the futures bases apply it.
- **API:**
- Engine state: `IsWarm`, `Atr`, `VolumeMean`, `VolumeSigma`, `Candle` (ECandle flags), `Has(ECandle)`, `Changed`, `LastBarTime`, `LastBar`.
- Bar events: `NewPivotHigh`, `NewPivotLow`, `BosUp`, `BosDown`, `ChochUp`, `ChochDown`.
- Pivots and structure: `LastHigh(layer = 0)`, `LastLow(layer = 0)`, `High(layer, ago)`, `Low(layer, ago)`, `PivotCount(layer, isHigh)`, `Structure(layer = 0)` → `EStructure {Unknown, Up, Down, Range}`, `DoubleTop(layer)`, `DoubleBottom(layer)`.
- `_Pivot` fields: `isHigh`, `layer`, `seq`, `price`, `extremeTime`, `extremeBar`, `confirmTime`, `confirmBar`, `label` (EPivotLabel), `isEqual`, `equalToSeq`, `legSizeAtr`.
- Lookup: `PriceActionFor(int dataStream, string role = null)`.
- **Timing:** steps on its own stream's closed bars. A pivot exists only from `confirmTime` (after `strength` bars) and is never relabelled. The engine resets on a roll.
- **Journal:**
- `PA{ds} L{layer} PivotHigh|PivotLow #{seq} {price} {label} [EQ#n] ext={t} conf={t};`
- `… BreakoutUp|BreakoutDown L{layer} of #{seq} {price} at {t};`

- `… BOS up|down [CHoCH] at {t};`
- `… roll reset at {t};`
- `PA{ds}-{role} created strength() layers() shared()`.
- **Tests:** `tests/PriceActionTests` (`McShim.cs`).
- **Snippet:**

```
PriceActionConfig.ApplyPreset(EPriceActionPreset.Futures); PriceActionConfig.enabled = true;   // Create()
// CalcBar
_PriceActionEngine pa = PriceActionEngine;
if (pa != null && pa.IsWarm && pa.BosUp && pa.Structure(1) == EStructure.Up)
{
    _Pivot hl = pa.LastLow(0);
    double stop = hl != null ? hl.price - 2 * tickSize() :
double.NaN;   // tickSize(): protected helper of _CustomSignal
}
```

## 2.9 M-TOD `_TodNormaliser`

- **Purpose:** time-of-day norms (z-score, median, percentile, cumulative z) for volume, delta, range and true range per session minute bucket.
- **Enable:** `TodMetrics = ETodMetric.Volume | ETodMetric.Range`.
- **Defaults:** `TodSessions 20`, `TodMinSessions 10`, `TodBucketMinutes 0` (host: 0 = the Data1 minute bar size, else 1). **VERIFY:** the class field `bucketMinutes` defaults to 1 but is always overwritten by the host.
- **API:** `Valid(m)`, `Z(m)`, `Median(m)`, `Pct(m)`, `ZCum(m)`, `Sessions(m)`, `VolumePool(minuteOfSession)`.
- **Timing:** norms come from prior completed sessions. Half-days are excluded (via `todHalfDay`). Range and TR are dropped on the roll-splice bar. Delta requires the futures base order flow.
- **Journal:** `TOD delta tier change a -> b: norm reset at session {day}`.
- **Tests:** `tests/TodTests`.
- **Snippet:**

```
TodMetrics = ETodMetric.Volume;
_TodNormaliser tod = TodNormaliser;
bool climax = tod != null && tod.Valid(ETodMetric.Volume) && tod.Z(ETodMetric.Volume) > 3;
```

## 2.10 M-XMKT `_XmktLeg`

- **Purpose:** cross-market legs on extra chart streams (for example NQ on Data2): log returns, rolling correlation, beta and relative-strength z. Orders stay on Data1.
- **Enable:** `XmktLegs.Add(new XmktLegConfig { role = "NQ", dataStream = 2 })`.
- **Defaults:** `role "leg"`, `dataStream 2`, `maxStaleSec 0` (= 2× the leg bar; off for non-time bars), `returnBars 1`, `corrLength 60`, `rsLength 60`, `leaderSwing false`.
- **API:** `XmktLeg(role)` returns `_XmktLeg` with `Available`, `LastLegTime`, `LastClose`, `LogReturn`, `Data1LogReturn`, `Correlation`, `Beta`, `RsZ`. `XmktLeaderSwing(role)` returns a PA engine on the leg.
- **Timing:** the leg is fed its last closed bar with `Time <= Data1 time` (`htfMapped`). It becomes unavailable when stale. Leg rolls are not inherited from Data1.
- **Journal:** `XMKT {role} Data{n} available at {t} (staleness limit …)`; Info `XMKT leg … symbol() resolution() dataFeed() maxStale()`.
- **Tests:** `tests/XmktTests`.
- **Snippet:**

```
_XmktLeg nq = XmktLeg("NQ");
if (nq != null && nq.Available && nq.RsZ < -2) { /* ES strong vs NQ */ }
```

## 2.11 M-TICK `_TickStream`

- **Purpose:** per-Data1-bar tick statistics from a lazy DataLoader 1-tick Trade request: prints, max prints/s, inter-print median/P10, burst z, and the size histogram.
- **Enable:** `TickConfig.enabled = true`.

- **Defaults:** `historyTicks 2,000,000`, `realTime true`, `burstZ 3`, `burstNormLength 200`, `burstMinSamples 30`, `sizeBuckets {1,2,5,10,20,50}`, `largePrintSize 20`, `sizeOutputsVerified false`.
- **API:** `TickStats` (last closed bar, or null) is a `TickBarStats` with `valid`, `quality`, `prints`, `maxPrintsPerSecond`, `interPrintMedianMs`, `interPrintP10Ms`, `burstZ`, `burst`, `sizeHistogram`, `largePrints`, `volume`.
- **Timing:** aggregated over `(Time[1], Time[0]]` at bar close. Late ticks are dropped. The size fields stay NaN/null until `sizeOutputsVerified`.
- **Journal:**
- `TICK DataLoader 1-tick Trade request for {sym}, N ticks back, RT=…;`
- `TICK n late tick(s) dropped …;`
- `TICK data missing/late … outputs invalid;`
- `TICK data valid again ….`
- **VERIFY:** the `Bar` member names are read by reflection (`Time`, `Close`, `TotalVolume`/`Volume`/`Ticks`). This is flagged in the code as "verify in the editor".
- **Tests:** `tests/TickTests`.

## 2.12 M-MTF (`_MtfMath`, host helpers)

- **Purpose:** closed-bar-only access to higher timeframes of the same instrument.
- **Enable:** `HtfStreams = new[] { 2 }`. StartCalc throws if the stream is missing or carries another root.
- **API:**
- Host: `HtfClosedIndex(ds, ago = 0)`, `HtfClosed(ds, ago = 0)`, `HtfClosedBar(ds, ago, out time, out o, out h, out l, out c, out v)`, `htfMapped(ds, maxBack = 2)` (-1 = none), `HtfForming(ds)`.
- Statics: `_MtfMath.ClosedIndex`, `MapIndex`, `SymbolRoot`, `IsStale`.
- **Journal:** `MTF Data{n} first mapping {t} -> {htf t} (ago i, status s)` (once per session); `MTF forming-bar access on Data{n} (not closedBarsOnly)`.
- **VERIFY:** whether MC stamps HTF bars with their end time ("Probe P2" in the code).
- **Tests:** `tests/MtfTests`.
- **Snippet** (`_MtfProbe`):

```
HtfStreams = new[] { 2 }; base.StartCalc();
// CalcBar
int i = htfMapped(2);
if (i >= 0) { double h60 = BarsOfData(2).High[i]; }
```

## 2.13 Rolls (`_RollDetector`, `_RollWindow`)

- **Purpose:** detect contract splices on continuous unadjusted futures.
- **Enable:** `RollDetection = ERollDetection.Series | Table | SeriesOrTable` (default `Off`).
- **Settings:** `RollGapTicks` (8); `RollDates` (a HashSet of trading days for Table).
- **Series rule:** a trading-day open that gaps ≥ RollGapTicks from the prior close on days 1–21 of Mar/Jun/Sep/Dec, or a change of `Bars.Info.Name`.
- **API:** `rollOn(ds)`, `RollBars` (1-based Data1 bars), `IsRollWindow(tradingDay)` (roll session ±1; requires rings, FVG or RollDetection).
- **Journal:** Info `VPF-4 roll splice detected tradingDay() bar() name()`; Warning `RollDetection is Off on a futures symbol …`.
- **Tests:** `tests/RollTests`. Event roll dates: `_EventCalendar.RuleRollDate`/`RollDateFor`.

## 2.14 Bar rings and FVG (`_BarRing`, `_FvgDetector`, `LevelRef`)

- **Rings:**
- `requireRing(ds, bars, consumer)` (after `base.StartCalc()`), then `Ring(ds)` returns a `_BarRing` with `Capacity`, `Count`, `Added`, `Dropped`, `LastBarNumber`, `this[ago]`, `At(idx)`, `IndexAtOrBefore`, `FirstIndexAtOrAfter`, `Covers`, `Window(t0, t1, t0Exclusive)` and `Session(day, rthOnly)`.
- `RingBar` fields: `bar`, `time`, `tradingDay`, `o h l c v up down`, `rth`, `rollBar`.
- `ringBarOf(ds, ago)`.
- `BarRingConfig.capacityBars 0` (auto, capped at `MaxAutoBars` 40000).
- Journal: `RING Data{n} capacity() consumers()`.

- **FVG:**
- Enable with `Fvgs.enabled = true`.
- Defaults: `dataStream 1`, `minTicks 1`, `minAtrFrac 0`, `atrLen 14`, `crossSessions false`, `rthOnly false`, `maxLive 20`, `maxAgeSessions 5`, `fillOn EFvgFillOn.Close`.
- `FvgDetector` has `Live`, `NewThisBar`, `Atr`, `Nearest(price, dir)`, `ToLevel(f, startEdge, stream)`.
- `Fvg` fields: `id`, `dir`, `lo`, `hi`, `start`, `createdTime`, `createdDay`, `touched`, `filled`, `expired`, `invalid`, `fillPct`, `ageSessions`, `Size`, `IsLive`, `Tag` (`FVG:U|D:yyyyMMddHHmm`).
- Use `fvgLevel(f, startEdge)` to get a `LevelRef`.
- **`LevelRef`:**
- fields: `kind` (`ELevelKind`), `price`, `lo`, `hi`, `createdTime` (usable from), `sourceT0`/`sourceT1`, `dataStream`, `tag`, `quality`, `crossesRoll`, `isRollWindow`;
- methods: `Line(...)`, `Zone(...)`, `StudyType`, `StudyKey`.
- **Tests:** `tests/VpHelperTests`.

## 2.15 Volume-profile bases (`_VolumeProfileBase`, `_VolumeProfileFuturesBase`)

- **Purpose:** MC volume profiles with POC/VA/HVN/LVN, naked POCs, up to three VWAP slots, confluence zones and targets, and the Phase 3 helpers (range, weekly and bar ladders, profile stats). The futures base adds the CME clock, IB/RTH stats, VPF-5 order flow and the CME holiday hooks.
- **Enable:** derive from `_VolumeProfileFuturesBase`. These bases *do* expose `[Input]`s:
- `HvnPercentile .8`, `LvnPercentile .2`, `MinNodeTicks 3`, `TrackNakedPoc true`, `NakedPocTickTolerance 1`;
- `Vwap1..3 Type/Length/FromDate/DataStream` (Vwap1 = Session on DS1; 2 and 3 Disabled);
- `DevelopingSource`, `DevProfileSourceMode`, `DevProfileDataStream`, `ValueAreaMethod`, `LookaheadAudit false`.
- **Main API:**
- Profiles: `ExtraVolumeProfileForBar([ds,] barIndex)` (1-based), `RecentCompletedProfiles(ds, count, asOfBar)`, `NakedPocs(ds[, asOf])`, `IsNewVolumeProfile(...)`.
- Confluence: `GetConfluenceZones`, `GetTopConfluenceZones([n])`, `GetConfluenceTargets(price, EVpDirection[, n])`, `GetConfluenceTargetZones`.
- VWAP slots: `CurrentVwap1..3`, `CurrentVwap1Sigma`, `Vwap1Active`.
- Auction and value: `ValueRelationship`, `IsPoorHigh`/`IsPoorLow`, `HasUnfinishedAuction`, `SinglePrints`, `ProfileShape`, `ValueMigrationScore`/`ValueMigrationState`, `Composite(stream, n)`, `IsBalanced`, `BalanceRange`.
- Futures base profiles: `RthProfile()`, `GlobexProfile()`, `PriorRthProfile()`.
- Range profiles: `RangeProfile(stream, t0, t1)`, `RangeProfileAfter`, `RangeProfileBars`, `OpenRange`/`CloseRange`/`RangeHandleProfile`, `RangeStatsOf`, `RangeLevels`.
- Weekly: `Week(weeksAgo)`, `WeeklyComposite(n)`, `CompletedWeeks`, `WeekKey`, `WeekLevels`.
- Ladders: `Ladder(ago[, minQuality])`, `LadderAt(time)`, `HtfLadder(ds, ago)`, `LadderProfile(b)`.
- Other: `ProfileEntropy(e, mmN)`, `SessionBars(day, rthOnly)`.
- **Order flow (futures base):**
- Enable with `OrderFlowEnabled = true`; configure `FlowCfg` (`_OrderFlow.FlowConfig`) and `FlowTickSource`.
- API: `Flow`, `Footprint(barsAgo)`, `BarDelta`, `CumDelta`, `DiagonalImbalances`, `StackedImbalances`, `Absorption`.
- **Phase 3 configs:** `RangeProfiles`, `WeeklyProfiles`, `ProfileStats`, `BarLadders` (`enabled false` by default; for the defaults see the Part 1 table sources and the `configs` in `_RangeProfile`/`_WeeklyProfiles`/`_ProfileStats`/`_BarLadder`).
- **Journal:**
- `VPB-1 own profile: source() mode() approximate() valueArea() audit()`;
- `RP build {tag} Data{n} [t0..t1] bars(a/b) poc() va() approx() roll()`;
- `RP coverage …`;
- `LADDER HTF Data{n} {t} coverage() -> totals-only`.
- **Tests:** `tests/ProfileTests`, `tests/VpHelperTests`, `tests/OrderFlowTests`.
- **Note:** your strategies inherit `_CustomSignal` directly, so this chapter applies only if you move a strategy onto a VP base.

## 2.16 M-PORT publisher and `_PortfolioRiskManager` (`_PortMM`)

- **Purpose:** publish `pm.*` keys to `MyPortfolioData` in Portfolio Trader. A money-management signal then sizes and limits risk across strategies.

- **Enable:**
- The publisher is automatic in PT (`PortfolioPublish`).
- Override as needed: `PortfolioFamily`, `PortfolioScore`, `PortfolioHalted`, `PortfolioCluster` (default from the symbol root: ES→EQIDX...), `PortfolioRegime`.
- Apply `_PortfolioRiskManager` as the portfolio's Money Management signal. It has no `[Input]`s; subclass it and override `Config` or `ConfigPath` (JSON).
- **Keys (`_PortKeys`):**
- Identity: `pm.ver`, `pm.id`, `pm.family`, `pm.cluster`.
- Equity and returns: `pm.eq`, `pm.pnl`, `pm.ret`, `pm.uRet`, `pm.vol`.
- Risk: `pm.R$`, `pm.qtyCode`, `pm.riskNew`, `pm.riskOpen`, `pm.openR`.
- State: `pm.state`, `pm.side`, `pm.regime`, `pm.regimeP`, `pm.score`, `pm.stamp`, `pm.day`, `pm.pos`.
- Events: `pm.evt*`.
- Written by the MM: `pm.mult`, `pm.heat`, `pm.dd`, `pm.rho`.
- **`EPortState`:** `{Flat, Armed, Long, Short, Exiting, Halted}`.
- **`PortMmConfig` defaults:**
- Sizing: `sizing RiskParity`, `bookVolTarget 0.10`, `riskPerTrade 0.005`.
- Risk caps: `heatCap 0.04`, `clusterCap 0.02`, `rhoMerge 0.9`, `rhoWin 60`, `rhoMinDays 40`, `maxPositions 6`.
- Drawdown: `ddSteps {5%→0.5, 10%→0.25, 15%→0}`, `ddRecoverHighDays 20`.
- Loss stops and kills: `dailyLossStop 0.02`, `dailyLossFlatten 0.03`, `stratKillR -8`, `stratKillTrades 20`.
- Other: `staleBars 3`, `minContracts 1`, `idmCap 2.5`.
- **API in your strategy:** `portfolioArm(int side, double entry, double stop, int qtyCode)` (called by `sizeForRisk` for Data1), `portfolioStop(double)`.
- **Journal:** Info `PORT publish on: id() family() cluster()`; Journal `PORT book multiplier x at t`. MM lines go to Output (`PORT Warning/Info`).
- **Tests:** `tests/PortMMTests`. Live checks use `_TestPortArm` + `_TestPortMM`.

## 2.17 A2-33 level study (`_LevelStudy`) and Gate 0 (`_Gate0`)

- **Level study:**
- Enable with `LevelStudyConfig.enabled = true`.
- Defaults: `controls 20`, `distLo .9`, `distHi 1.1`, `reactionBars 60`, `sigmaBars 60`, `touchTicks 0`, `maxAge 15`, `deathR 1`, `seed 33`, `from`/`to`, `csvPath ""`, `controlsEitherSide false`, `reactionAfterTouch false`.
- Register prior-session levels on the session's first bar with `levelStudyAdd(type, price, ageSessions, quality = NaN, key = null)`.
- Results: Info `A2-33 RESULT events() types() window()` plus a summary per type at RUN-END, and an optional CSV.
- **VERIFY:** no strategy on `main` enables `LevelStudyConfig`. The code comment names `_Test1`, but `_Test1` is a plain `SignalObject`.
- **Gate 0** (`Gate0Engine`, `EGate0Mode/Family/Canary`, `Gate0Bar/Intent/Profile/Vwap/Atr/Session`): an MC-free research harness, hosted by `_TestGate0`. **VERIFY:** its host usage is not covered in this manual.
- **Tests:** `tests/LevelStudyTests`, `tests/Gate0Tests`.

## 2.18 Utility engines

- `_Gmm`: `Fit`, `Fit1D`, `FitK`, `FitOnce`, `LogDensity`, `Density1D`, `Order`, `ParamCount`; `GmmModel`, `GmmFitConfig`.
- `_Rng`: deterministic `Mix`, `Seed(params long[])`, `Hash(string)`.
- `_ProfileStats`: `Entropy`, `MmN`, `KL`, `KLKde*`, `KdeOnGrid`, `ParkinsonPerBar`, `ParkinsonSessionPrice`, `EwmaForecast`, `AggregateBars`, `WilderAtr`; `GmmSession.FromBars`.
- `_SignalJournal`: a CS-0 probe description only (`PROBE_ID`, `Describe()`).

---

# Appendix

### A. `ETraceLevel`

| Level | Value | Enables |
|---|---|---|
| None | 0 | nothing (the default after `Create`) |
| Error | 1 | errors (for example the Event coverage failure) |
| Warning | 2 | + one-time "engine queried but not enabled" warnings, config warnings |
| Info | 3 | + RUN / RUN-END, `entry blocked:`, roll detected, XMKT leg, PORT publish |
| Journal | 4 | + engine facts, `MCJournal_<runId>.csv`, `MCEvents_<runId>.csv` |
| Verbose | 5 | + equity and account details |
| Debug | 6 | + engine debug sinks (for example PA `trace`), sizing reasons |

## B. Files and formats

All files go to `%LOCALAPPDATA%\MCSignalLogs\` (fallback: `%TEMP%`).

- **Trace log** `MCSignal{yyyyMMdd-HHmmss}-{pid}.csv`, one per process. Columns: `level,yyyy-MM-dd HH:mm:ss {zone},signalName,symbol,runId,message`.
- **Trade journal** `MCJournal_<runId>.csv`, with the header: `runId,strategy,symbol,tag,side,qty,entryTime,entryPrice,exitTime,exitPrice,exitOrder,barsHeld,profit,commission,a` plus `,evtDayTier,evtMinSince` when M-EVENT is on. The `context` column holds the `journalArm` pairs as `k=v|k=v`.
- **Signal events** `MCEvents_<runId>.csv`: `runId,strategy,symbol,tag,direction,time,bar,price,stop,riskPerUnit,taken,controlMode`, plus the MFE/MAE R columns per horizon (VERIFY the exact suffix names in `signalEventsStep`). The context is `k=v;`.
- **Engine line prefixes:** `DAILY`, `REGIME`, `REGGATE`, `EVENT`, `AV`, `PA{ds}[-role]`, `TOD`, `XMKT`, `TICK`, `MTF`, `RING`, `RP`, `LADDER`, `VPB-1`, `VPF-4`, `VPF-7`, `PORT`, `A2-33`.

## C. Running the tests

All tests need the .NET 8 SDK. Each `tests/<Name>/run.sh` does `dotnet run -c Release` in its folder. Where a `gen.sh` exists, it generates `Generated/*.cs` from the real study sources (for example it extracts the `// <calendar-rules …>` blocks), and the build runs it itself.

```
cd MultiCharts.Net
for t in tests/*/run.sh; do echo "== $t"; bash "$t" || break; done
bash tests/RegimeTests/run.sh          # also supports the reference.py export path
bash tests/EventTests/gen.sh            # regenerate only
```

Suites: Avwap, Calendar, Daily, Event, Gate0, Gmm, LevelStudy, Mtf, OrderFlow, PortMM, PriceAction, Profile, RegimeGate, Regime, Roll, SessionClock, Tick, Tod, VpHelper, Xmkt. `gen.sh` exists in Calendar, Daily, Event, Gate0, Gmm, OrderFlow, Profile and RegimeGate.

## D. Glossary

- **AsOf**: the date whose completed data produced a value. It is usable strictly after that date.
- **AVWAP**: anchored VWAP from an event bar.
- **BOS / CHoCH**: break of structure / change of character (a BOS against the prior structure).
- **Closed bar**: `Bars.Status == EBarState.Close`. Engines commit only on these.
- **Globex date**: the CME trading date. The 18:00 ET evening session belongs to the next date.
- **Half-day**: a CME early RTH close (`RthCloseEt`, 13:15 ET) or a holiday halt.
- **HTF**: a higher-timeframe stream of the same instrument (`HtfStreams`).
- **Leg**: a cross-market stream (`XmktLegs`).

• **No-RTH day**: Globex is open but there is no cash session (an NYSE holiday).
• **Placebo (P1/P2)**: an AVWAP twin with neutralised volume, used for falsification.
• **Roll splice**: the first bar of a new contract on a continuous unadjusted series.
• **RTH value window**: `(09:30, 16:00]` ET, or up to the early close.
• **Tier**: event importance (T1 > T2 > T3, C = calendar).
• **TOD**: time-of-day normalisation per session-minute bucket.